

Scoped Verification for Reliable Long-Horizon Agentic Context Evolution under Distribution Shift

Dan C. Hsu

*RedMind Research, San Francisco, CA, USA
National Taiwan University, Taipei, Taiwan*

DAN@REDMINDRESEARCH.ORG

Luke Lu

RedMind Research, San Francisco, CA, USA

LUKE@REDMINDRESEARCH.ORG

Code: <https://github.com/RedMind-Research/GRACE>

Abstract

Deployed LLM agents rely on agentic context, the model-external textual control content assembled by an operational harness. In this work, the mutable component of that context is a persistent system-level instruction that is updated from operational experience while the model, tools, and harness remain fixed. Over long evolution horizons, flat-text maintenance makes verification increasingly difficult as accumulated instructions grow and interact. We propose Graph-Regularized Agentic Context Evolution (GRACE), which maintains the persistent instruction component as a typed semantic graph and validates proposed updates within the local typed neighborhoods of modified nodes. Accepted graph updates are reconstructed as incremental edits to the textual instruction checkpoint used at deployment. We evaluate GRACE within a fixed telecom agent harness derived from τ^2 -bench under a controlled distribution-shift protocol. Across five independent replications, GRACE improves strict reliability, measured by pass³, from the Gemini 2.5 Flash zero-shot value of 0.091 to 0.673 ± 0.136 at the final checkpoint. This exceeds a Gemini 3.1 Pro zero-shot reference of 0.242 on the same held-out set, while the flat-text HCE baseline finishes at 0.191 ± 0.051 . These results identify two requirements for reliable long-horizon context evolution, a structural substrate that makes verification local and a consolidation mechanism that keeps accumulated instruction content usable.

Keywords: agentic context, context evolution, persistent instruction, structural validation, reliability, distribution shift

1. Introduction

As large language models have become better at following natural-language instructions, model-external text has become a primary control surface for deployed agents (Khattab et al., 2024; Zhang et al., 2026). In operational agent systems, this text is assembled with task inputs, tool observations, and harness-provided information before each model call (Yao et al., 2025; Barres et al., 2025). We refer to the resulting inference-time input as the agent context. In the setting studied here, the mutable component of this context is a persistent system-level instruction that specifies the agent’s role, behavioral constraints, procedural guidance, and domain assumptions (Qin et al., 2025; Lee et al., 2024). When task distributions shift, this persistent instruction can encode stale or incomplete assumptions about the environment it is meant to govern. We study context evolution as the process of revising this persistent context component from operational experience while the model, tools, and harness remain fixed. The evolving artifact may be a playbook (Zhang et al., 2026), an adaptive memory (Suzgun et al., 2026), a set of strategic principles (Wu et al.,

2025), or a set of structured guidelines (Pei et al., 2025). Across these forms, prior work has shown that iterative updates can improve agent behavior (Zhang et al., 2026; Shinn et al., 2023).

However, over extended evolution horizons, previously accumulated improvements can be undermined by inconsistencies introduced in subsequent steps. Because the persistent instruction is included in the agent context throughout deployment, degradation in this artifact can compromise operational reliability as well as update efficiency. Context collapse during full-document rewriting has been documented as one failure mode in agentic context engineering (Zhang et al., 2026), alongside unbounded memory growth in adaptive memory (Suzgun et al., 2026) and guideline conflict accumulation in prompt evolution (Pei et al., 2025). These failures point to a shared structural limitation. Flat-text maintenance leaves relationships among rules, constraints, and procedures implicit in the document’s linear order, which makes verification increasingly dependent on long-context processing as the artifact grows (Liu et al., 2024). Prior work has shown that model-external guidance can be iteratively improved, but it has not directly isolated whether verification remains effective when the persistent instruction grows over long evolution horizons.

We propose Graph-Regularized Agentic Context Evolution (GRACE), which maintains a typed semantic graph as the intermediate substrate for evolving the persistent instruction component of agent context. GRACE represents atomic instruction units as graph nodes, encodes structural relationships among them as typed directed edges, and uses graph locality to validate each proposed update near the affected nodes. Accepted graph updates are reconstructed as incremental text edits to the deployed instruction checkpoint. This paper studies the representation substrate within a fixed deployment interface. The agent model, tools, harness, diagnosis procedure, and evaluation set are held fixed, while the representation and verification mechanism vary. We evaluate GRACE in the telecom customer-service domain derived from τ^2 -bench (Barres et al., 2025), using its operational harness with fixed tool interfaces and evaluation criteria. Under a controlled shift protocol across 10 evolution batches, the main comparison uses three conditions with the same diagnosis procedure but different representation and verification mechanisms: GRACE, GRACE without structural analysis, and HCE. Across replicated runs, GRACE remains ahead at the later checkpoints, indicating that structure-enabled verification and consolidation both matter for sustained context evolution.

In summary, the paper makes three contributions. First, GRACE introduces a graph-regularized substrate for evolving the persistent instruction component of agent context and performs scoped structural validation at each evolution step. Second, the evaluation protocol introduces alternating distribution shift, so the same trajectory tests both within-phase improvement and cross-phase retention. Third, replicated experiments in the telecom domain show that GRACE outperforms a flat-text context-evolution baseline and a graph-based ablation without structural analysis. The ablation separates two failure modes. Contradiction avoidance helps, but sustained improvement also requires consolidation of the growing instruction substrate.

2. Related Work

Context evolution and persistent control artifacts. Optimizing the textual artifacts that steer LLM behavior has progressed from prompt-level refinement to long-horizon context evolution. Prompt optimization and self-refinement methods such as OPRO (Yang et al., 2023), TextGrad (Yuksekgonul et al., 2025), DSPy (Khatab et al., 2024), Reflexion (Shinn et al., 2023), and Self-Refine (Madaan et al., 2023) demonstrate effective short-horizon improvement, but they do not maintain a persistent context component through repeated batch-level updates and address a different optimization setting. More recent work treats model-external guidance as a persistent artifact that accumulates knowledge across episodes. ACE (Zhang et al., 2026) introduces incremental delta updates to prevent context collapse, SCOPE (Pei et al., 2025) applies conflict resolution and subsumption pruning on dual-stream memory, and Dynamic Cheatsheet (Suzgun et al., 2026) and ExpeL (Zhao et al., 2024) accumulate reusable guidance and experiential insights over time. These systems are closest in spirit because they manage growing model-external guidance, but they differ in artifact interface, update contract, and evaluation protocol. We focus on a narrower question inside one fixed harness, namely whether the representation substrate used by the evolution operator can sustain verification as the persistent instruction component grows.

Broader agent-evolution frameworks address tool, workflow, or environment adaptation (Xi et al., 2025; Liu et al., 2026). A-Evolve (Lin et al., 2026) frames cross-episode updates over a persistent artifact state. Our work shares this cross-episode perspective, but focuses specifically on the persistent system-level instruction inside agent context and on controlled instantiations of the same evolution interface. ProEvolve (Li et al., 2026) likewise applies typed-graph transformations, but to evolve benchmark environments, whereas GRACE applies graph-structured editing to the persistent instruction itself. Complementarily, Meta-Harness (Lee et al., 2026) optimizes the harness code surrounding the model, whereas we hold the harness fixed and evolve only the instruction component it assembles. Across the context-evolution methods above, the evolving artifact is typically maintained as flat text, and the question of whether an alternative representation substrate can sustain verification effectiveness over extended horizons remains underexplored.

Structured knowledge representation for LLMs. Graph structures have been widely adopted in LLM systems for knowledge organization, memory, and retrieval. GraphRAG (Edge et al., 2025) constructs entity-relation graphs to support query-focused summarization over large corpora, HippoRAG (Gutiérrez et al., 2024) builds knowledge graphs as long-term memory for associative retrieval, and broader surveys map the expanding landscape of knowledge-graph and LLM integration (Pan et al., 2024). These applications demonstrate that graphs can organize knowledge that LLMs operate over, but their use as an evolving substrate for persistent instruction content remains underexplored. GRACE addresses this gap by using a typed semantic graph as the intermediate representation on which context evolution and structural validation operate.

3. Problem Formulation

We study long-horizon agentic context evolution under a fixed operational harness. Let $\pi_\theta(a \mid c)$ denote the frozen action-selection policy induced by the LLM, tools, and harness

when supplied with agent context c . The model parameters, tool interfaces, harness logic, and context-assembly procedure are held fixed throughout the evolution trajectory.

In this work, the mutable component of the agent context is a persistent system-level instruction, denoted by ℓ_t after the t -th batch-level update. This instruction is included in the context supplied to the agent, but it is not itself the action-selection policy. At inference time, the harness assembles a task-specific context

$$c_t(h, x) = \text{Assemble}(\ell_t, h, x), \quad (1)$$

where h denotes harness-provided information and x denotes the task or user input. The full task-specific context changes across episodes through h and x , while only ℓ_t is updated across evolution batches.

We model context evolution as a closed-loop cross-episode update process, following the persistent artifact update perspective of Lin et al. (2026). We use *substrate* to denote the mutable representation used during offline evolution. The substrate is the internal form in which the persistent instruction is maintained, edited, and validated before the next instruction checkpoint is reconstructed. Let s_t denote this substrate at update step t . The initial instruction ℓ_0 is shared across all conditions, and the initial substrate s_0 is initialized from ℓ_0 . For text-maintained methods, s_t is the instruction text itself. For GRACE, $s_t = G_t$ is a graph-regularized substrate from which ℓ_t is reconstructed.

For each update step $t \in \{1, \dots, T\}$, the agent is executed on tasks drawn from the batch distribution $\mathcal{D}_t$. Execution uses the frozen action-selection policy π_θ under contexts assembled from the previous instruction checkpoint ℓ_{t-1} . The resulting interaction traces are

$$\mathcal{E}_t = F_{\text{Experience}}(\pi_\theta, \text{Assemble}(\ell_{t-1}, \cdot, \cdot), \mathcal{D}_t). \quad (2)$$

The diagnostic observation is then computed as

$$o_t = F_{\text{Diagnose}}(\mathcal{E}_t). \quad (3)$$

The evolution function uses the diagnostic observation to update the substrate and produce the next instruction checkpoint:

$$(s_t, \ell_t) \leftarrow F_{\text{Evolve}}(s_{t-1}, \ell_{t-1}, o_t). \quad (4)$$

For a text-maintained context-evolution baseline,

$$\ell_t \leftarrow F_{\text{Evolve}}^{\text{text}}(\ell_{t-1}, o_t). \quad (5)$$

For GRACE,

$$(G_t, \ell_t) \leftarrow F_{\text{Evolve}}^{\text{graph}}(G_{t-1}, \ell_{t-1}, o_t). \quad (6)$$

The graph G_t is used only during offline editing and validation. Deployment uses the reconstructed textual instruction ℓ_t , which is assembled with h and x to form the agent context supplied to π_θ .

We hold $F_{\text{Experience}}$ and F_{Diagnose} fixed across all conditions. The comparison therefore isolates how each method maintains, validates, and reconstructs the evolving instruction component through F_{Evolve} . In this work, GRACE is the proposed graph-regularized realization of this operator, introduced in §4. The empirical protocol in §5.1 specifies the

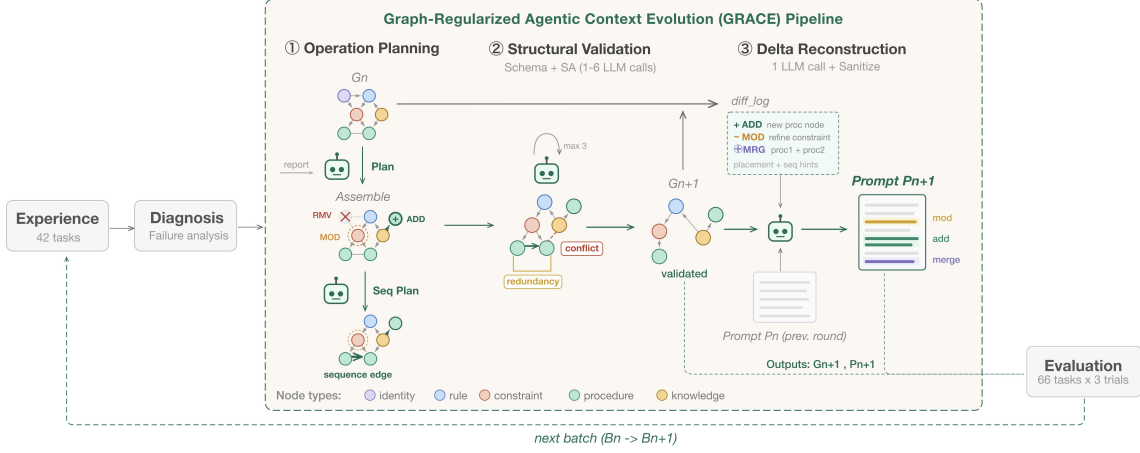


Figure 1: **Overview of the GRACE evolution pipeline.** Given a diagnosis report and the current graph G_{t-1} , the pipeline produces an updated graph G_t and corresponding instruction checkpoint ℓ_t through three stages: Operation Planning, Structural Validation, and Delta Reconstruction. Sequence planning is performed as a substep within Operation Planning, while Structural Validation combines deterministic schema checks with iterative structural analysis (SA).

controlled shift schedule, checkpoint evaluation procedure, and reliability metrics used to evaluate the resulting deployed context checkpoints. Throughout the paper, context evolution refers to the batch-level process that updates the persistent instruction component of the agent context. The term policy is reserved for the frozen action-selection policy $\pi_\theta(a | c)$, except when discussing terminology used in prior work.

4. Method: Graph-Regularized Agentic Context Evolution

In the notation of §3, GRACE instantiates the evolution substrate s_t as a typed graph G_t . Nodes encode atomic instruction units extracted from the persistent system-level instruction ℓ_t , and edges encode structural relations among those units. The deployed artifact remains textual. After schema-constrained editing and local structural validation, GRACE reconstructs the next instruction checkpoint ℓ_t , which is assembled into the agent context at inference time.

4.1. Graph-Regularized Substrate

Following the formulation of heterogeneous information networks (Sun et al., 2011), GRACE maintains the substrate as a directed typed graph $G_t = (V_t, E_t)$ at update step t , where nodes encode atomic instruction units and edges encode directed structural relations. The graph is equipped with an object-type mapping function $\varphi : V \rightarrow \mathcal{A}$ and a relation-type mapping function $\psi : E \rightarrow \mathcal{R}$. GRACE instantiates the object and relation type sets as

$$\mathcal{A} = \{\text{identity, norm, knowledge}\}, \quad \mathcal{R} = \{\text{supports, refines, sequence}\}. \quad (7)$$

Following the ontology-specification convention of Gruber (1993), we specify each object and relation type by an identifier, a natural-language description of its intended meaning,

and constraints that determine admissible instances. These definitions induce the network schema

$$T_G = (\mathcal{A}, \mathcal{R}, \{\sigma_r\}_{r \in \mathcal{R}}), \quad (8)$$

where each $\sigma_r \subseteq \mathcal{A} \times \mathcal{A}$ specifies the admissible source and target object types for relation type r . For every edge $e = (u, v)$ with $\psi(e) = r$, GRACE requires $(\varphi(u), \varphi(v)) \in \sigma_r$, ensuring schema-conformant updates. For GRACE, the initial substrate s_0 is the graph G_0 , obtained by a semantics-preserving decomposition of the shared initial instruction ℓ_0 into schema-conformant objects and relations. Figure 2 visualizes the schema used in our implementation.

The object types correspond to classes in the Ontolingua convention, treated as unary relations whose instance variable ranges over admissible graph nodes. Relation-type definitions constrain source and target arguments and therefore determine the endpoint signatures in σ_r . Table 1 summarizes the object and relation type definitions used by GRACE, and Table 2 gives the induced type-signature matrix.

Kind	Symbol	Role	Necessary condition
Object	identity	Agent standing	Content fixes the agent’s mandate and the subject to which conduct is attributed.
Object	norm	Conduct rule	Content states a standard the agent’s action is required to satisfy or breach.
Object	knowledge	Domain premise	Content states what is the case in the task domain and serves as a reasoning premise.
Relation	supports	Grounding	$(\varphi(u), \varphi(v)) \in \sigma_{\text{supports}}$ and u grounds, justifies, or reinforces v .
Relation	refines	Specialization	$\varphi(u) = \varphi(v)$; u is a narrower case of v obtained by adding conditions, exceptions, or operational detail without contradiction.
Relation	sequence	Procedure order	$\varphi(u) = \varphi(v) = \text{norm}$ and u precedes v in procedural execution order.

Table 1: **Object and relation type definitions in the GRACE network schema.** Each row gives the natural-language rendering of the corresponding `:def` condition.

Source type	identity	norm	knowledge
identity	refines	supports	$\emptyset$
norm	$\emptyset$	supports, refines, sequence	$\emptyset$
knowledge	$\emptyset$	supports	supports, refines

Table 2: **Type-signature matrix for the GRACE network schema.** Each cell lists relation types allowed from the source object type to the target object type.

4.2. Evolution Pipeline

At each update step, GRACE instantiates the evolution operator in Eq. 4 as a mapping from the previous graph substrate, previous instruction checkpoint, and diagnostic observation to the next graph substrate and instruction checkpoint, $(G_{t-1}, \ell_{t-1}, o_t) \mapsto (G_t, \ell_t)$. The pipeline obtains these outputs through operation planning, structural validation, and delta reconstruction.

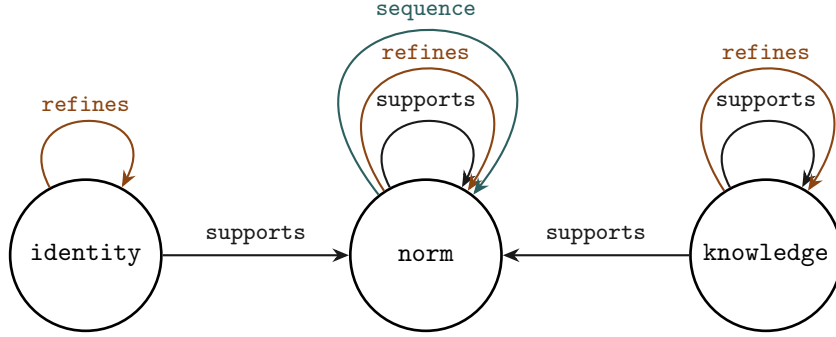


Figure 2: **GRACE network schema** $T_G = (\mathcal{A}, \mathcal{R}, \{\sigma_r\}_{r \in \mathcal{R}})$. Nodes denote object types in $\mathcal{A}$, and directed labeled edges denote admissible relation types: **supports**, **refines**, and **sequence**.

Operation planning. The first stage converts the diagnosis into schema-constrained graph updates. Let each graph state be represented as (V, E, φ, ψ, m) , where $m : V \rightarrow \mathcal{T}$ maps each node to its natural-language instruction content. GRACE defines a finite editing algebra over T_G :

$$\mathcal{O}_{T_G} = \{\text{AddNode}, \text{AddEdge}, \text{ModifyNode}, \text{RemoveNode}, \text{RemoveEdge}, \text{Merge}\}. \quad (9)$$

An operation belongs to $\mathcal{O}_{T_G}$ only when applying it preserves schema conformance under T_G . Table 3 summarizes the six operators. An LLM-mediated edit planner with a fixed prompt interface proposes a finite set of typed graph edits over this algebra:

$$\Delta_t^{\text{edit}} = F_{\text{edit}}(G_{t-1}, o_t), \quad \Delta_t^{\text{edit}} \subseteq \mathcal{O}_{T_G}. \quad (10)$$

Operator	Arguments	Description
AddNode	(a, m_v)	Introduces node v with $\varphi(v) = a$ and content $m(v) = m_v$, where $a \in \mathcal{A}$.
AddEdge	(u, v, r)	Introduces edge $e = (u, v)$ with $\psi(e) = r$ when $r \in \mathcal{R}$ and $(\varphi(u), \varphi(v)) \in \sigma_r$.
ModifyNode	(v, m'_v)	Replaces $m(v)$ with m'_v while preserving $\varphi(v)$.
RemoveNode	(v)	Removes v and all incident edges from the graph.
RemoveEdge	(e)	Removes an existing edge e from the graph.
Merge	(u, v, m_w)	For same-type nodes, replaces u and v with representative node w and reroutes admissible incident edges under T_G .

Table 3: **Graph editing operators in $\mathcal{O}_{T_G}$.**

Each edit in Δ_t^{edit} is traceable to a diagnosis finding and must conform to T_G . For every proposed relation edge $e = (u, v)$ with $\psi(e) = r$, the source and target object types must satisfy $(\varphi(u), \varphi(v)) \in \sigma_r$. The planned edits Δ_t^{edit} are deterministically applied to G_{t-1} , yielding an intermediate graph $\hat{G}_t$ together with the edit-touched node set U_t^{edit} . The intermediate graph is then passed to a relation-maintenance module before structural validation. This LLM-mediated pass reviews the typed relation layer of $\hat{G}_t$, focusing on the

relations incident to the edit-touched set U_t^{edit} , and proposes relation-level edits:

$$\Delta_t^{\text{rel}} = F_{\text{relation}}(\hat{G}_t, U_t^{\text{edit}}), \quad \Delta_t^{\text{rel}} \subseteq \mathcal{O}_{T_G}. \quad (11)$$

Relation-level edits must also conform to T_G . The accepted relation edits Δ_t^{rel} are deterministically incorporated into $\hat{G}_t$, yielding the candidate graph G_t^{cand} together with the relation-touched set U_t^{rel} .

Structural validation. The candidate graph G_t^{cand} is first checked for conformance to the network schema. Structural analysis (SA) then examines the affected typed neighborhoods for local semantic issues, and flagged issues are repaired through schema-conformant operations. Following the PathSim framework of [Sun et al. \(2011\)](#), where meta-paths define semantically meaningful comparison contexts for objects in heterogeneous information networks, structural analysis adapts the same typed-neighborhood principle to LLM-mediated instruction validation over local typed graph neighborhoods. Given a symmetric meta-path $\mathcal{P}$, PathSim between same-type objects x and y is

$$s_P(x, y) = \frac{2 \times |\{p_{x \rightsquigarrow y} : p_{x \rightsquigarrow y} \in \mathcal{P}\}|}{|\{p_{x \rightsquigarrow x} : p_{x \rightsquigarrow x} \in \mathcal{P}\}| + |\{p_{y \rightsquigarrow y} : p_{y \rightsquigarrow y} \in \mathcal{P}\}|}. \quad (12)$$

Here $p_{x \rightsquigarrow y}$ denotes a path instance between x and y following $\mathcal{P}$. GRACE adopts PathSim’s typed comparison principle while changing the objective from similarity ranking to scoped validation. A typed local neighborhood defines which instruction units are compared and which structural evidence can distinguish, merge, or repair them.

Let $U_t = U_t^{\text{edit}} \cup U_t^{\text{rel}}$ denote the update-touched node set, and let d_G denote shortest-path distance in graph G . For any validation radius k , we define the k -hop typed neighborhood as

$$N_k(U_t, G_t^{\text{cand}}) = \left\{ v \in V_t^{\text{cand}} : \min_{u \in U_t} d_{G_t^{\text{cand}}}(u, v) \leq k \right\}. \quad (13)$$

Here, k -hop refers to all nodes reachable from U_t within at most k graph hops. In implementation, structural analysis uses a deterministic progressive schedule that starts from $k = 3$ and increases the radius by 3 across a fixed number of local expansion rounds. Let k^* denote the final radius reached by this schedule. The validation neighborhood H_t is the local typed subgraph induced by $N_{k^*}(U_t, G_t^{\text{cand}})$ inside G_t^{cand} , containing the selected neighborhood nodes, all candidate-graph edges among them, and the inherited type mappings. Structural analysis is performed on H_t , and repairs are applied back to G_t^{cand} to obtain G_t .

Structural analysis detects two semantic issue families. First, contradiction captures joint unsatisfiability under overlapping applicability scope. For nodes $u, v \in V(H_t)$, let $\text{Overlap}_{H_t}(u, v)$ indicate that the two units apply under overlapping graph-local scope, and let $\text{Mod}_{H_t}(x)$ denote the set of coherent instruction interpretations satisfying content x in the validation neighborhood H_t . We define

$$\text{Contr}_{H_t}(u, v) \iff \text{Overlap}_{H_t}(u, v) \wedge \text{Mod}_{H_t}(m(u) \wedge m(v)) = \emptyset. \quad (14)$$

This follows the standard view that contradictory contents cannot be jointly satisfied, adapted to natural-language instruction units whose applicability is determined by the validation neighborhood ([de Marneffe et al., 2008](#)).

Second, redundancy captures lack of marginal semantic contribution within the same object type. We first define a redundancy candidate:

$$\text{RedCand}_{H_t}(u, v) \iff \varphi(u) = \varphi(v) \wedge \text{Overlap}_{H_t}(u, v) \wedge (\text{Entails}_{H_t, \varphi(u)}(u, v) \vee \text{Entails}_{H_t, \varphi(u)}(v, u)). \quad (15)$$

The predicate $\text{Entails}_{H_t, a}(u, v)$ denotes type-conditioned semantic subsumption for object type $a \in \mathcal{A}$: under the validation neighborhood H_t , the content of v can be inferred from the content of u . This follows textual-entailment theory (Dagan et al., 2009; Androustopoulos and Malakasiotis, 2010) and operationalizes redundancy as informational equivalence or subsumption between texts (Zanzotto et al., 2011). The same-type restriction prevents identity, norm, and knowledge nodes from being collapsed merely because their natural-language contents are related. A candidate becomes an actionable redundancy issue only when the graph provides no structural reason to keep the two units distinct:

$$\text{Red}_{H_t}(u, v) \iff \text{RedCand}_{H_t}(u, v) \wedge \neg \text{Diff}_{H_t}(u, v). \quad (16)$$

Here, $\text{Diff}_{H_t}(u, v)$ indicates that the validation neighborhood structurally differentiates the two units. Differentiating evidence can include specialization, grounding, procedural ordering, or distinct typed support neighborhoods, preventing semantically related but functionally distinct instruction units from collapsing. Structural analysis flags pairs that satisfy either contradiction or actionable redundancy:

$$\mathcal{I}_t = \{(u, v) \in V(H_t) \times V(H_t) : \text{Contr}_{H_t}(u, v) \vee \text{Red}_{H_t}(u, v)\}. \quad (17)$$

Detected issues in $\mathcal{I}_t$ are repaired through schema-conformant operations in $\mathcal{O}_{T_G}$, resulting in the validated graph G_t .

Delta reconstruction After validation, delta reconstruction converts the graph-level change $\log \mathcal{L}_t^G = G_t \triangle G_{t-1}$ into the deployed instruction checkpoint, where $\triangle$ denotes the component-wise symmetric difference over the typed graph state, interpreted from G_{t-1} to G_t . The instruction checkpoint is produced as $\ell_t = F_{\text{recon}}(\ell_{t-1}, \mathcal{L}_t^G)$, editing only text regions aligned with validated graph-level changes while preserving unaffected content, formatting, and section structure. At deployment, ℓ_t is assembled with harness-provided information and task input to form the agent context supplied to π_θ .

5. Experiments

5.1. Experimental Setup

Benchmark, harness, and comparison scope. We evaluate on the telecom domain of τ^2 -bench (Barres et al., 2025), which comprises 2,285 composable tasks constructed from 15 atomic subtask groups across three intent categories. Within this harness, the evolving component is the persistent system-level instruction included in the agent context. All conditions share the same initial instruction, experience batches, diagnosis procedure, held-out evaluation set, agent model, and user simulator. They differ only in how F_{Evolve} maintains, validates, and reconstructs this persistent context component.

	MMS	Mobile Data	Service	Total
Evaluation	30 (10/10/10)	30 (10/10/10)	6 (2/2/2)	66
Exp. Phase A (B0,1,4,5,8,9)	30 (10/10/10)	9 (3/3/3)	3 (1/1/1)	42
Exp. Phase B (B2,3,6,7)	9 (3/3/3)	30 (10/10/10)	3 (1/1/1)	42

Table 4: **Controlled shift protocol and task allocation.** The held-out evaluation set is shared across all checkpoints and never used for evolution. Experience batches alternate between two intent-mixture phases, both containing all three intents. MMS and Mobile Data are reweighted in opposite directions, while Service is held constant. Numbers in parentheses denote Easy/Hard/None customer persona splits. No two batches share any task.

Controlled shift protocol and task allocation. To evaluate whether the evolved instruction component preserves reliable behavior when the experience distribution changes, we construct a controlled shift protocol over user intents. The protocol preserves subpopulation support across phases while changing relative intent frequencies, matching the subpopulation-shift setting of Koh et al. (2021). We partition 486 tasks into a held-out evaluation set of 66 tasks and 10 experience batches of 42 tasks each, disjoint at the task level but sharing underlying atomic subtask groups. The experience batches alternate between two intent-mixture configurations across the 10-batch trajectory, as detailed in Table 4. The supplementary material quantifies the induced shift over intents and atomic subtask groups.

Evaluation metrics. At each reported evaluation checkpoint, the agent is evaluated using the deployed context assembled from instruction checkpoint ℓ_t on the same 66-task held-out set with three independent trials per task, yielding 198 episodes per checkpoint. The replicated evaluation reports the shared initial instruction ℓ_0 and later checkpoints $\ell_6, \ell_8, \ell_{10}$. Following τ -bench (Yao et al., 2025), $\text{pass}@k$ measures task-level capability under a best-of- k criterion, while $\hat{\text{pass}}@k$ measures trial-level reliability under an all- k criterion. To measure retention under phase changes, we report intent-level backward transfer following Lopez-Paz and Ranzato (2017). For a phase transition from X to Y and an intent d not emphasized in Phase Y , $\text{BWT}_d(X \rightarrow Y) = \hat{\text{pass}}@k_d(\pi_{Y_{\text{end}}}) - \hat{\text{pass}}@k_d(\pi_{X_{\text{end}}})$. Positive BWT indicates retained or improved performance on that intent after the experience distribution shifts away from it. In all reported BWT results, $k = 3$. The post-hoc consistency audit used for diagnostic analysis is described in the supplementary material.

Evolution conditions and shared diagnosis. We compare three primary instantiations of F_{Evolve} . *Holistic Context Evolution (HCE)* applies a single-pass text-maintained update to the persistent instruction with a built-in consistency guard. *GRACE without SA* ablates structural analysis from the graph-regularized instruction substrate, while *GRACE* denotes the full method. These are controlled instantiations of the same evolution interface, designed to isolate the effects of representation and structural validation, not to reimplement full prior systems with different artifacts and protocols. All conditions use Gemini 2.5 Flash as the agent model and GPT-4.1 as the user simulator. They share the same two-stage diagnosis procedure, which reflects on failed conversations and synthesizes the

resulting findings into prioritized update themes. The procedure is fixed across conditions, but diagnosis content may differ after the first update. The comparison matches the initial instruction, experience batches, diagnosis procedure, held-out evaluation set, and one batch-level evolution opportunity per update, but not internal LLM-call budgets. Zero-shot Gemini 2.5 Flash and Gemini 3.1 Pro references are included only as fixed held-out references. The evolution comparison itself is between the three Gemini 2.5 Flash conditions above.

Replication and audit scope. Each primary condition is run for five independent evolution trajectories. At every reported checkpoint, each trajectory is evaluated on the same 66 held-out tasks with three trials per task. Each method-checkpoint mean therefore aggregates five 198-episode evaluations. HCE receives the same diagnosis signal and update schedule as GRACE and maintains the deployed instruction directly as text with an explicit consistency guard, so it is not a zero-shot prompt baseline. The post-hoc LLM-as-Judge audit (Zheng et al., 2023) is applied only to $\ell_0, \ell_6, \ell_8, \ell_{10}$ and is used only for mechanism analysis. It is not used to select checkpoints, modify instruction checkpoints, or compute pass^3 , $\text{pass}@3$, or $\text{pass}@1$. The supplementary material gives the controlled-shift quantification and audit protocol.

5.2. Main Results

Table 5 summarizes the checkpoint results, and Figure 3 shows the replicated pass^3 trajectories. The conditions share the agent model, simulator, experience batches, diagnosis procedure, held-out set, and update schedule. The comparison therefore turns on how each instantiation of F_{Evolve} maintains, verifies, and reconstructs the evolving instruction component.

GRACE sustains cumulative improvement. Across five independent replications, GRACE is the only primary condition that shows sustained improvement through the later checkpoints of the controlled shift protocol (Figure 3). Starting from the shared initial instruction with pass^3 of 0.091, it reaches 0.530 ± 0.086 at ℓ_6 , 0.594 ± 0.135 at ℓ_8 , and 0.673 ± 0.136 at ℓ_{10} . Its $\text{pass}@3$ remains near ceiling over the same checkpoints, reaching 0.979 ± 0.025 at ℓ_{10} (Table 5). The remaining gap between $\text{pass}@3$ and pass^3 indicates that capability is easier to recover than strict reliability, which requires all three independent trials to succeed.

Flat-text instruction maintenance exhibits distinct failure patterns. HCE improves over the shared initial instruction but does not sustain the gain. It reaches 0.215 ± 0.066 at ℓ_6 and 0.236 ± 0.109 at ℓ_8 , then finishes at 0.191 ± 0.051 . The same pattern is visible in $\text{pass}@3$. HCE reaches 0.618 ± 0.218 at ℓ_6 but drops to 0.479 ± 0.244 at ℓ_{10} . This trajectory illustrates that flat-text context evolution can exploit diagnostic signal but remains vulnerable to later degradation as the maintained instruction grows.

Graph structure alone is not sufficient for sustained improvement. GRACE without SA improves to 0.458 ± 0.177 at ℓ_6 , but the gain is not consolidated. Its pass^3 drops to 0.303 ± 0.165 at ℓ_8 and 0.248 ± 0.144 at ℓ_{10} , while $\text{pass}@3$ falls from 0.845 ± 0.189 to 0.685 ± 0.254 . The graph substrate makes the instruction artifact more verifiable, but the ablation shows that structure alone does not protect accumulated gains over time.

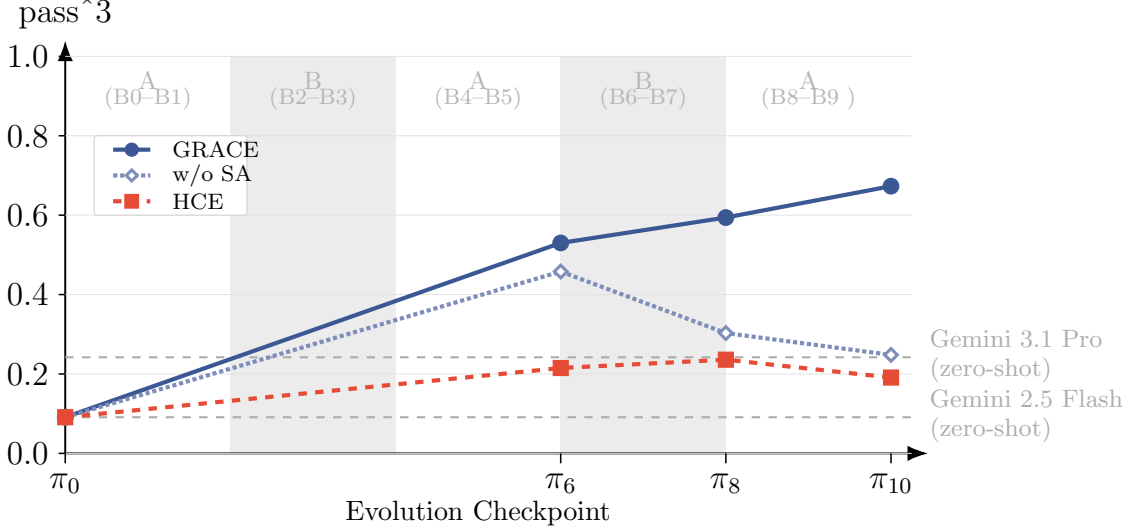


Figure 3: **Replicated pass³ checkpoint trajectories for the three primary conditions.** Points report mean pass³ over five independent replications. Standard deviations are reported in Table 5. Dashed horizontal lines mark the zero-shot pass³ references for Gemini 3.1 Pro and Gemini 2.5 Flash on the same held-out set.

Metric	Method	ℓ_6	ℓ_8	ℓ_{10}
pass ³	GRACE	.530 (.086)	.594 (.135)	.673 (.136)
	w/o SA	.458 (.177)	.303 (.165)	.248 (.144)
	HCE	.215 (.066)	.236 (.109)	.191 (.051)
pass@3	GRACE	.942 (.075)	.936 (.071)	.979 (.025)
	w/o SA	.845 (.189)	.670 (.221)	.685 (.254)
	HCE	.618 (.218)	.418 (.286)	.479 (.244)
pass@1	GRACE	.757 (.078)	.775 (.109)	.851 (.073)
	w/o SA	.662 (.192)	.490 (.196)	.468 (.199)
	HCE	.400 (.116)	.327 (.201)	.330 (.138)

Table 5: **Replicated checkpoint summary.** Values are mean (std) over five runs. The shared initial instruction has pass³ = .091, pass@3 = .333, and pass@1 = .202.

5.3. Analysis

The three trajectories differ not only in final score but also in failure mode. We next examine the mechanisms behind those differences.

Contradiction accumulation distinguishes HCE from the other failure modes.

We use the post-hoc LLM-as-Judge audit described in §5.1 to examine whether evolved instruction artifacts accumulate internal contradictions. The audit uses a single contradiction category. Two instruction statements are counted as contradictory when they prescribe incompatible behavior under overlapping conditions and the graph or text provides no structural relation that resolves the tension. Across the audited checkpoints $\ell_0, \ell_6, \ell_8, \ell_{10}$,

Method	ℓ_0	ℓ_6	ℓ_8	ℓ_{10}
GRACE	23,363	42,902	46,471	51,413
w/o SA	23,363	45,207	49,343	54,893
HCE	23,363	59,668	73,822	82,592

Table 6: **Mean deployed instruction size in characters.** Values average over five runs and measure the persistent system-level instruction assembled into the deployed agent context, not the internal graph substrate.

HCE has the highest contradiction count at 3.35 on average, while GRACE and GRACE without SA remain in a narrower range from 1.75 to 1.90. This evidence is correlational, but it matches the observed performance collapse. As HCE’s flat text grows, its built-in consistency guard has increasing difficulty keeping incompatible rules apart.

GRACE without SA keeps contradiction counts low but still fails to sustain improvement. Avoiding contradictions is therefore not enough. The remaining failure mode is lack of consolidation over the growing graph substrate.

Graph structure suppresses contradictions but does not consolidate. GRACE and GRACE without SA maintain similarly low contradiction counts, indicating that typed graph representation and incremental delta reconstruction already help avoid gross contradictions. The difference appears in growth and consolidation. By the final checkpoint, the graph-only ablation reaches 421 nodes, compared to 370 nodes for GRACE. The deployed instruction-size measurements in Table 6 show the same pattern at the text layer on the reported checkpoints. HCE grows substantially more than either graph-based condition, while GRACE remains slightly smaller than the graph-only ablation at ℓ_{10} . The LLM-as-Judge redundancy audit points in the same direction. Across $\ell_0, \ell_6, \ell_8, \ell_{10}$, GRACE without SA has a higher redundancy count than GRACE on average, 4.85 compared to 4.15, and reaches 6.5 at ℓ_{10} . This expansion matches the stochastic trajectory in §5.2. Individual checkpoints can improve, but the gains are not consolidated into a durable instruction component. The graph and SA play different roles. The graph makes relationships among accumulated instruction units explicit and helps prevent gross contradictions. SA consolidates the growing substrate by merging overlapping content and resolving subtle tensions.

Retention under distribution shift reflects these mechanism differences. The controlled shift protocol tests whether these mechanism differences translate into retention under changing experience distributions. GRACE shows positive or zero backward transfer in all cycles except Mobile Data in Cycle 1, where a temporary drop of 0.233 is later recovered. HCE exhibits persistent negative backward transfer, with Mobile Data BWT reaching -0.500 in Cycle 1 and never recovering. The mechanism analysis explains the split. HCE’s contradiction-laden instruction artifact loses coherence when the experience distribution shifts, while GRACE’s consolidated instruction component is more resilient to the shift.

6. Conclusion

This paper studied long-horizon agentic context evolution through the lens of verification. In the setting considered here, the evolving object is the persistent system-level instruction

assembled into the agent context, while the model, tools, and harness remain fixed. GRACE maintains this instruction component as a typed semantic graph, enabling structural validation to focus on affected typed neighborhoods without rereading the entire flat-text artifact at each update. Under a controlled shift protocol across 10 evolution batches, replicated evaluation shows that GRACE sustains later-checkpoint improvement more reliably than both HCE and GRACE without SA. The ablation clarifies the mechanism. Contradiction avoidance helps, but durable improvement also requires active consolidation of the growing instruction substrate.

The evidence is still limited to the telecom domain of τ^2 -bench with a fixed agent model and harness. Future work should evaluate GRACE across additional domains and against full prior context-evolution systems under matched implementation budgets. Within the controlled substrate-level setting studied here, the main lesson is direct. Long-horizon context evolution is easier to verify when relationships among accumulated instruction units are explicit and local enough to inspect.

References

- Ion Androutsopoulos and Prodrimos Malakasiotis. A survey of paraphrasing and textual entailment methods. *Journal of Artificial Intelligence Research*, 38:135–187, 2010. doi: 10.1613/jair.2985. URL <https://jair.org/index.php/jair/article/view/10651>.
- Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2 -bench: Evaluating conversational agents in a dual-control environment, 2025. URL <https://arxiv.org/abs/2506.07982>.
- Ido Dagan, Bill Dolan, Bernardo Magnini, and Dan Roth. Recognizing textual entailment: Rational, evaluation and approaches. *Natural Language Engineering*, 15(4):i–xvii, 2009. doi: 10.1017/S1351324909990209.
- Marie-Catherine de Marneffe, Anna N. Rafferty, and Christopher D. Manning. Finding contradictions in text. In Johanna D. Moore, Simone Teufel, James Allan, and Sadaoki Furui, editors, *Proceedings of ACL-08: HLT*, pages 1039–1047, Columbus, Ohio, June 2008. Association for Computational Linguistics. URL <https://aclanthology.org/P08-1118/>.
- Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph rag approach to query-focused summarization, 2025. URL <https://arxiv.org/abs/2404.16130>.
- Thomas R. Gruber. A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2):199–220, 1993. ISSN 1042-8143. doi: <https://doi.org/10.1006/knac.1993.1008>. URL <https://www.sciencedirect.com/science/article/pii/S1042814383710083>.
- Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. Hipporag: Neurobiologically inspired long-term memory for large language mod-

- els. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 59532–59569. Curran Associates, Inc., 2024. doi: 10.52202/079017-1902. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/6ddc001d07ca4f319af96a3024f6dbd1-Paper-Conference.pdf.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan A, Saiful Haq, Ashutosh Sharma, Thomas Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. Dspy: Compiling declarative language model calls into state-of-the-art pipelines. In B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun, editors, *International Conference on Learning Representations*, volume 2024, pages 54928–54958, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/f1cf02ce09757f57c3b93c0db83181e0-Paper-Conference.pdf.
- Pang Wei Koh, Shiori Sagawa, Henrik Marklund, Sang Michael Xie, Marvin Zhang, Akshay Balsubramani, Weihua Hu, Michihiro Yasunaga, Richard Lanus Phillips, Irena Gao, Tony Lee, Etienne David, Ian Stavness, Wei Guo, Berton Earnshaw, Imran Haque, Sara M Beery, Jure Leskovec, Anshul Kundaje, Emma Pierson, Sergey Levine, Chelsea Finn, and Percy Liang. Wilds: A benchmark of in-the-wild distribution shifts. In Marina Meila and Tong Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 5637–5664. PMLR, 18–24 Jul 2021. URL <https://proceedings.mlr.press/v139/koh21a.html>.
- Seongyun Lee, Sue Hyun Park, Seungone Kim, and Minjoon Seo. Aligning to thousands of preferences via system message generalization. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 73783–73829. Curran Associates, Inc., 2024. doi: 10.52202/079017-2347. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/86c9df30129f7663ad4d429b6f80d461-Paper-Conference.pdf.
- Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses, 2026. URL <https://arxiv.org/abs/2603.28052>.
- Guangrui Li, Yaochen Xie, Yi Liu, Ziwei Dong, Xingyuan Pan, Tianqi Zheng, Jason Choi, Michael J. Morais, Binit Jha, Shaunak Mishra, Bingrou Zhou, Chen Luo, Monica Xiao Cheng, and Dawn Song. The world won’t stay still: Programmable evolution for agent benchmarks, 2026. URL <https://arxiv.org/abs/2603.05910>.
- Minhua Lin, Hanqing Lu, Zhan Shi, Bing He, Rui Mao, Zhiwei Zhang, Zongyu Wu, Xianfeng Tang, Hui Liu, Zhenwei Dai, Xiang Zhang, Suhang Wang, Benoit Dumoulin, and Jian Pei. Position: Agentic evolution is the path to evolving llms, 2026. URL <https://arxiv.org/abs/2602.00359>.
- Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranajpe, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics*, 12:157–173, 2024.

- Zichen Liu, Anya Sims, Keyu Duan, Changyu Chen, Simon Yu, Xiangxin Zhou, Haotian Xu, Shaopan Xiong, Bo Liu, Chenmien Tan, Chuen Yang Beh, Weixun Wang, Hao Zhu, Weiyan Shi, Diyi Yang, Michael Shieh, Yee Whye Teh, Wee Sun Lee, and Min Lin. Gem: A gym for agentic llms, 2026. URL <https://arxiv.org/abs/2510.01051>.
- David Lopez-Paz and Marc' Aurelio Ranzato. Gradient episodic memory for continual learning. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/f87522788a2be2d171666752f97ddeb-Paper.pdf.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 46534–46594. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/91edff07232fb1b55a505a9e9f6c0ff3-Paper-Conference.pdf.
- Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 36(7):3580–3599, 2024. doi: 10.1109/TKDE.2024.3352100.
- Zehua Pei, Hui-Ling Zhen, Shixiong Kai, Sinno Jialin Pan, Yunhe Wang, Mingxuan Yuan, and Bei Yu. Scope: Prompt evolution for enhancing agent effectiveness. *arXiv preprint arXiv:2512.15374*, 2025.
- Yanzhao Qin, Tao Zhang, Yanjun Shen, Wenjing Luo, sunhaoze, Yan Zhang, Yujing Qiao, Weipeng Chen, Zenan Zhou, Wentao Zhang, and Bin CUI. Sysbench: Can llms follow system message? In Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu, editors, *International Conference on Learning Representations*, volume 2025, pages 74442–74466, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/file/b917f916e7eed84ffe8f5e63492b2be8-Paper-Conference.pdf.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: language agents with verbal reinforcement learning. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/1b44b878bb782e6954cd888628510e90-Paper-Conference.pdf.
- Yizhou Sun, Jiawei Han, Xifeng Yan, Philip S. Yu, and Tianyi Wu. Pathsim: meta path-based top-k similarity search in heterogeneous information networks. *Proc. VLDB Endow.*, 4(11):992–1003, August 2011. ISSN 2150-8097. doi: 10.14778/3402707.3402736. URL <https://doi.org/10.14778/3402707.3402736>.

- Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. Dynamic cheatsheet: Test-time learning with adaptive memory. In Vera Demberg, Kentaro Inui, and Lluís Marquez, editors, *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7080–7106, Rabat, Morocco, March 2026. Association for Computational Linguistics. ISBN 979-8-89176-380-7. doi: 10.18653/v1/2026.eacl-long.333. URL <https://aclanthology.org/2026.eacl-long.333/>.
- Rong Wu, Xiaoman Wang, Jianbiao Mei, Pinlong Cai, Daocheng Fu, Cheng Yang, Licheng Wen, Xuemeng Yang, Yufan Shen, Yuxin Wang, et al. Evolver: Self-evolving llm agents through an experience-driven lifecycle. *arXiv preprint arXiv:2510.16079*, 2025.
- Zhiheng Xi, Yiwen Ding, Wenxiang Chen, Boyang Hong, Honglin Guo, Junzhe Wang, Xin Guo, Dingwen Yang, Chenyang Liao, Wei He, Songyang Gao, Lu Chen, Rui Zheng, Yicheng Zou, Tao Gui, Qi Zhang, Xipeng Qiu, Xuanjing Huang, Zuxuan Wu, and Yungang Jiang. AgentGym: Evaluating and training large language model-based agents across diverse environments. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 27914–27961, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.1355. URL <https://aclanthology.org/2025.acl-long.1355/>.
- Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *The Twelfth International Conference on Learning Representations*, 2023.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu, editors, *International Conference on Learning Representations*, volume 2025, pages 9965–10017, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/file/1b126cc38b8638e07bef37e7b2bb72bf-Paper-Conference.pdf.
- Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. Optimizing generative ai by backpropagating language model feedback. *Nature*, 639(8055):609–616, 2025.
- Fabio Massimo Zanzotto, Marco Pennacchiotti, and Kostas Tsioutsoulis. Linguistic redundancy in Twitter. In Regina Barzilay and Mark Johnson, editors, *Proceedings of the 2011 Conference on Empirical Methods in Natural Language Processing*, pages 659–669, Edinburgh, Scotland, UK., July 2011. Association for Computational Linguistics. URL <https://aclanthology.org/D11-1061/>.
- Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=eC4ygDs02R>.

- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners. In *Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence and Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence and Fourteenth Symposium on Educational Advances in Artificial Intelligence*, AAAI’24/IAAI’24/EAAI’24. AAAI Press, 2024. ISBN 978-1-57735-887-9. doi: 10.1609/aaai.v38i17.29936. URL <https://doi.org/10.1609/aaai.v38i17.29936>.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 46595–46623. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/91f18a1287b398d378ef22505bf41832-Paper-Datasets_and_Benchmarks.pdf.